National University of Sciences and Technology
(NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing
Class: **BSCS-2023-CD**
Course: **CS-346 Compiler Construction**

Lab 10: Code Optimization

Date: 16 April 2026

Name: Abrar Butt
Qalam ID: 460297

Lab Instructor: Ms. Urooj Akmal
Instructor: Miss Yusra

GitHub Link for code files given below:
https://github.com/Abrarbut/compiler-construction.git

Task 1	Code Optimization (Sloppy vs Optimized)
Task 2	Unreachable Code Detection via Control Flow

NUST SEECS, H-12, Islamabad, Pakistan

Contents

1 Introduction **2**
1.1 Objectives . . . 2
1.2 Tools / Software Requirements . . . 2

2 Background: Optimization Techniques **2**

3 Task 1 – Code Optimization **3**
3.1 Overview . . . 3
3.2 Sloppy Program (`sloppy.c`) . . . 3
3.3 Optimized Program (`optimized.c`) . . . 5
3.4 Optimization Summary Table . . . 6
3.5 Program Output . . . 7

4 Task 2 – Unreachable Code Detection via Control Flow **8**
4.1 Algorithm . . . 8
4.2 CFG for `sloppy.c` . . . 8
4.3 Source Code (`unreachable_detector.c`) . . . 9
4.4 Program Output . . . 13
4.5 Analysis of Results . . . 15

5 Compilation Instructions **15**

6 Conclusion **15**

1 Introduction

Code optimization is a program transformation technique that improves code efficiency by reducing resource consumption (CPU time, memory usage) while increasing execution speed. During optimization, high-level programming constructs are transformed into more efficient low-level representations.

Three principles of effective compiler optimization:

1. The optimized code must **preserve** the original program's semantics.
2. It must **enhance** execution speed and/or reduce resource usage.
3. The optimization process itself must be **efficient** and not delay compilation.

1.1 Objectives

1. Understand and implement code-optimization techniques used in compiler construction to improve program performance *without* altering its output.

1.2 Tools / Software Requirements

- GCC (GNU Compiler Collection) – Linux / Windows

2 Background: Optimization Techniques

Table 1 summarizes the eight techniques demonstrated in this lab.

Table 1: Common Code Optimization Techniques

#	Technique	Description
1	Constant Folding	Evaluate constant expressions at compile time.
2	Constant Propagation	Substitute known constant values into uses.
3	Common Subexpression Elim.	Compute a repeated expression once and reuse.
4	Strength Reduction	Replace expensive ops with cheaper equivalents.
5	Dead Code Elimination	Remove variables/statements whose results are unused.
6	Algebraic Simplification	Apply identity laws: $x \times 1 = x$, $x + 0 = x$, etc.
7	Loop Invariant Code Motion	Move loop-invariant computations outside the loop.
8	Copy Propagation	Replace copies with their original source variable.

3 Task 1 – Code Optimization

3.1 Overview

A "sloppy" C program (`sloppy.c`) is constructed that contains intentional inefficiencies illustrating all eight techniques above. An optimized version (`optimized.c`) applies each technique with explanatory comments. **Both programs produce identical output.**

3.2 Sloppy Program (`sloppy.c`)

sloppy.c – Unoptimized Program (9 inefficiency groups)

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    // ===== 1. CONSTANT FOLDING opportunities =====
    int token_count = 100;
    int line_count = token_count;
    int buffer_size = 8 * 1024;          // Should be 8192
    int max_tokens = 50 * 2;             // Should be 100
    int hash_prime = 7 * 11 * 13;        // Should be 1001

    // ===== 2. CONSTANT PROPAGATION opportunities =====
    int symbols = token_count / 10;      // token_count always 100 -> 10
    int keywords = token_count / 25;     // Should be 4
    int identifiers = token_count - symbols - keywords; // 100-10-4=86

    // ===== 3. COMMON SUBEXPRESSION ELIMINATION =====
    int tc = token_count;
    int lc = line_count;
    int total_a = (tc + lc) * 2;         // (100+100)*2 = 400
    int total_b = (tc + lc) * 2;         // Repeated computation!
    int total_c = (tc + lc) * 1;         // Different multiplier
    int total_d = (tc + lc) * 1;         // Another repeat!

    // ===== 4. STRENGTH REDUCTION opportunities =====
    int mem_a = token_count * 4;         // Could be << 2
    int mem_b = token_count * 8;         // Could be << 3
    int mem_c = token_count * 16;        // Could be << 4
    int mem_d = token_count * 32;        // Could be << 5

    // ===== 5. DEAD CODE (unused variables) =====
    int dead_var1 = 9999;                // Never used
    int dead_var2 = 7777;                // Never used
    dead_var1 = dead_var2 + 100;         // Still unused

    // ===== 6. ALGEBRAIC SIMPLIFICATION =====
    int alg_a = token_count * 1;         // Just token_count
    int alg_b = token_count + 0;         // Just token_count
    int alg_c = token_count / 1;         // Just token_count
    int alg_d = token_count * 0;         // Just 0
```

```c
    int alg_e = token_count - 0;          // Just token_count

    // ===== 7. LOOP INVARIANT CODE MOTION =====
    int parse_sum = 0;
    for(int i = 0; i < 10; i++) {
        // max_tokens*2 computed every iteration (100*2=200)
        parse_sum += (max_tokens * 2) + i;  // Should hoist 200
    }

    // ===== 8. COPY PROPAGATION =====
    int cp_x = token_count;               // cp_x = 100
    int cp_y = cp_x;                      // cp_y = 100
    int cp_z = cp_y + 50;                 // cp_z = 150 (direct: token_count+50)

    // ===== UNREACHABLE CODE (for Task 2) =====
    int flag = 1;
    if(flag == 1) {
        goto end_processing;
    }
    // These statements are unreachable!
    printf("NEVER prints!\n");
    symbols *= 999;
    keywords += 500;

end_processing:
    // Output results
    printf("===== Compiler Analysis Report =====\n");
    printf("Token Count     : %d\n", token_count);
    printf("Line Count      : %d\n", line_count);
    printf("Buffer Size     : %d\n", buffer_size);
    printf("Max Tokens      : %d\n", max_tokens);
    printf("Hash Prime      : %d\n", hash_prime);
    printf("Symbols         : %d\n", symbols);
    printf("Keywords        : %d\n", keywords);
    printf("Identifiers     : %d\n", identifiers);
    printf("Total A         : %d\n", total_a);
    printf("Total B         : %d\n", total_b);
    printf("Total C         : %d\n", total_c);
    printf("Total D         : %d\n", total_d);
    printf("Mem A (x4)      : %d\n", mem_a);
    printf("Mem B (x8)      : %d\n", mem_b);
    printf("Mem C (x16)     : %d\n", mem_c);
    printf("Mem D (x32)     : %d\n", mem_d);
    printf("Alg A           : %d\n", alg_a);
    printf("Alg B           : %d\n", alg_b);
    printf("Alg C           : %d\n", alg_c);
    printf("Alg D           : %d\n", alg_d);
    printf("Alg E           : %d\n", alg_e);
    printf("Parse Sum       : %d\n", parse_sum);
    printf("CP Z            : %d\n", cp_z);
    printf("=====================================\n");

    return 0;
}
```

3.3 Optimized Program (optimized.c)

optimized.c – Optimized Program (8 techniques applied)

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    // ===== 1. CONSTANT FOLDING (pre-computed at compile time) =====
    int token_count = 100;
    int line_count = token_count;          // Constant propagation will help
    int buffer_size = 8192;                // 8*1024 folded
    int max_tokens = 100;                  // 50*2 folded
    int hash_prime = 1001;                 // 7*11*13 folded

    // ===== 2. CONSTANT PROPAGATION (substitute known values) =====
    int symbols = 10;                      // token_count/10 = 10
    int keywords = 4;                      // token_count/25 = 4
    int identifiers = 86;                  // 100-10-4 = 86

    // ===== 3. COMMON SUBEXPRESSION ELIMINATION =====
    int tc = token_count;
    int lc = line_count;
    int sum_tc_lc = tc + lc;               // Compute once!
    int total_a = sum_tc_lc * 2;           // 400
    int total_b = sum_tc_lc * 2;           // Reuse sum_tc_lc
    int total_c = sum_tc_lc * 1;           // Reuse sum_tc_lc
    int total_d = sum_tc_lc * 1;           // Reuse sum_tc_lc

    // ===== 4. STRENGTH REDUCTION (multiplication to shift) =====
    int mem_a = token_count << 2;          // token_count * 4
    int mem_b = token_count << 3;          // token_count * 8
    int mem_c = token_count << 4;          // token_count * 16
    int mem_d = token_count << 5;          // token_count * 32

    // ===== 5. DEAD CODE ELIMINATION (removed dead_var1, dead_var2) =====
    // (Variables that are never used are simply omitted)

    // ===== 6. ALGEBRAIC SIMPLIFICATION =====
    int alg_a = token_count;               // token_count * 1
    int alg_b = token_count;               // token_count + 0
    int alg_c = token_count;               // token_count / 1
    int alg_d = 0;                         // token_count * 0
    int alg_e = token_count;               // token_count - 0

    // ===== 7. LOOP INVARIANT CODE MOTION =====
    int parse_sum = 0;
    int loop_invariant = max_tokens * 2;   // Hoisted out of loop (200)
    for(int i = 0; i < 10; i++) {
        parse_sum += loop_invariant + i;   // Use hoisted value
    }
```

```c

    // ===== 8. COPY PROPAGATION (eliminate intermediate copies) =====
    int cp_z = token_count + 50;            // Direct, no cp_x or cp_y needed

    // ===== UNREACHABLE CODE ANALYSIS (for Task 2) =====
    int flag = 1;
    if(flag == 1) {
        goto end_processing;
    }
    // Unreachable blocks identified and removed by CFG analysis

end_processing:
    // Output results (identical to sloppy.c)
    printf("===== Compiler Analysis Report =====\n");
    printf("Token Count     : %d\n", token_count);
    printf("Line Count      : %d\n", line_count);
    printf("Buffer Size     : %d\n", buffer_size);
    printf("Max Tokens      : %d\n", max_tokens);
    printf("Hash Prime      : %d\n", hash_prime);
    printf("Symbols         : %d\n", symbols);
    printf("Keywords        : %d\n", keywords);
    printf("Identifiers     : %d\n", identifiers);
    printf("Total A         : %d\n", total_a);
    printf("Total B         : %d\n", total_b);
    printf("Total C         : %d\n", total_c);
    printf("Total D         : %d\n", total_d);
    printf("Mem A (x4)      : %d\n", mem_a);
    printf("Mem B (x8)      : %d\n", mem_b);
    printf("Mem C (x16)     : %d\n", mem_c);
    printf("Mem D (x32)     : %d\n", mem_d);
    printf("Alg A           : %d\n", alg_a);
    printf("Alg B           : %d\n", alg_b);
    printf("Alg C           : %d\n", alg_c);
    printf("Alg D           : %d\n", alg_d);
    printf("Alg E           : %d\n", alg_e);
    printf("Parse Sum       : %d\n", parse_sum);
    printf("CP Z            : %d\n", cp_z);
    printf("=====================================\n");

    return 0;
}
```

3.4 Optimization Summary Table

Table 2 maps each optimization to the concrete before/after code change.

Table 2: Optimization Techniques Applied in `optimized.c`

OPT	Technique	Before (sloppy)	After (optimized)
1	Constant Folding	`8 * 1024`	`8192`
1	Constant Folding	`7 * 11 * 13`	`1001`
2	Constant Propagation	`token_count / 10`	`10`
3	CSE	`(tc+lc)*2` (x2 times)	`sum_tc*2`; reuse `total_a`
4	Strength Reduction	`token_count * 4`	`token_count << 2`
5	Dead Code Elimination	`dead_var1`, `dead_var2`	removed entirely
6	Algebraic Simplification	`token_count * 1`	`token_count`
6	Algebraic Simplification	`token_count * 0`	`0`
7	Loop Invariant Code Motion	`max_tokens*2` inside loop	200 hoisted out
8	Copy Propagation	`cp_y=cp_x; cp_z=cp_y+50`	`cp_z=token_count+50`

3.5 Program Output

Both programs produce exactly the same output:

```
===== Compiler Analysis Report =====
Token Count     : 100
Line Count      : 100
Buffer Size     : 8192
Max Tokens      : 100
Hash Prime      : 1001
Symbols         : 10
Keywords        : 4
Identifiers     : 86
Total A         : 400
Total B         : 400
Total C         : 100
Total D         : 100
Mem A (x4)      : 400
Mem B (x8)      : 800
Mem C (x16)     : 1600
Mem D (x32)     : 3200
Alg A           : 100
Alg B           : 100
Alg C           : 100
Alg D           : 0
Alg E           : 100
Parse Sum       : 2045
CP Z            : 150
=====================================
```

Listing 1: Identical output from sloppy.c and optimized.c

Verification: Compile and run both programs with
```
gcc sloppy.c -o sloppy && gcc optimized.c -o optimized
./sloppy > out1.txt && ./optimized > out2.txt && diff out1.txt
out2.txt
```
The `diff` produces **no output**, confirming identical results.

4 Task 2 – Unreachable Code Detection via Control Flow

4.1 Algorithm

1. **Build a Control Flow Graph (CFG):** represent each group of sequential statements as a *basic block* (node). Add directed edges for every possible control transfer (sequential, branch true, branch false, goto).
2. **Depth-First Search (DFS) from the entry block B0:** mark every visited block as *reachable*.
3. **Report unreachable blocks:** any block not marked after the DFS has no path from the entry – it is dead code.
4. **Remove dead blocks:** emit the optimized CFG containing only the reachable blocks.

4.2 CFG for `sloppy.c`

The 15 basic blocks of `sloppy.c` and their successor edges are:

Table 3: Control Flow Graph of `sloppy.c`

Block	Label	Code Summary	Successor(s)
B0	ENTRY	`int token_count=100, lc=tc`	B1
B1	CONST_FOLD	`buffer=8*1024, max=50*2 ...`	B2
B2	CONST_PROP	`symbols=tc/10, kw=tc/25`	B3
B3	CSE	`total_a=(tc+lc)*2` x4	B4
B4	STRENGTH	`mem_a=tc*4, mem_b=tc*8 ...`	B5
B5	DEAD_CODE	`dead_var1=9999; dead_var2`	B6
B6	ALG_SIMP	`alg_a=tc*1 ... alg_e=tc/1`	B7
B7	LOOP_BODY	`for i=0..9: parse_sum+=...`	B8
B8	COPY_PROP	`cp_x=tc; cp_y=cp_x; cp_z`	B9
B9	BRANCH	`if(flag==1) goto end_proc`	B12
red!15 B10	UNREACH_1	`printf("NEVER prints!")`	B11
red!15 B11	UNREACH_2	`symbols*=999; kw+=500`	B12
B12	END_PROC	`end_processing: label`	B13
B13	OUTPUT	`printf(all results)`	B14
B14	EXIT	`return 0`	–

Observation: B10 and B11 have **no incoming edges** because `flag = 1` is a compile-time constant, making the `if(flag==1)` branch always taken to B12. A DFS from B0 never visits B10 or B11.

4.3 Source Code (`unreachable_detector.c`)

```c
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX_BLOCKS 20
#define MAX_NAME 20

typedef struct {
    int id;
    char label[MAX_NAME];
    char code[100];
    int successors[MAX_BLOCKS];
    int num_successors;
```

```
    bool reachable;
} BasicBlock;

BasicBlock blocks[MAX_BLOCKS];
int num_blocks = 0;

void add_block(int id, const char* label, const char* code) {
    blocks[num_blocks].id = id;
    strcpy(blocks[num_blocks].label, label);
    strcpy(blocks[num_blocks].code, code);
    blocks[num_blocks].num_successors = 0;
    blocks[num_blocks].reachable = false;
    num_blocks++;
}

void add_successor(int from_id, int to_id) {
    for(int i = 0; i < num_blocks; i++) {
        if(blocks[i].id == from_id) {
            blocks[i].successors[blocks[i].num_successors++] = to_id;
            return;
        }
    }
}

void dfs(int block_id) {
    for(int i = 0; i < num_blocks; i++) {
        if(blocks[i].id == block_id) {
            if(blocks[i].reachable) return;
            blocks[i].reachable = true;
            for(int j = 0; j < blocks[i].num_successors; j++) {
                dfs(blocks[i].successors[j]);
            }
            return;
        }
    }
}

void print_cfg() {
    printf("\n=== Original Control Flow Graph (CFG) ===\n");
    printf("+-----+------------+------------------------------+---------------------+\n");
    printf("| ID  | Label      | Code                         | Successors          |\n");
    printf("+-----+------------+------------------------------+---------------------+\n");
    for(int i = 0; i < num_blocks; i++) {
        printf("| B%d  | %-10s | %-29s ", blocks[i].id, blocks[i].label, blocks[i].code);
        if(blocks[i].num_successors > 0) {
            printf("| B%d", blocks[i].successors[0]);
            for(int j = 1; j < blocks[i].num_successors; j++) {
                printf(",B%d", blocks[i].successors[j]);
            }
            printf("%-*s", 17 - (blocks[i].num_successors * 2), " ");
```

```
        } else {
            printf("| EXIT%-18s", " ");
        }
        printf("|\n");
    }
    printf("+-----+------------+------------------------------+--------------------+\n");
}

void print_reachability() {
    printf("\n=== Reachability Analysis Results ===\n");
    printf("+-----+------------+------------------------------+--------------------+\n");
    printf("| ID  | Label      | Code                         | Status             |\n");
    printf("+-----+------------+------------------------------+--------------------+\n");
    int reachable_count = 0, unreachable_count = 0;
    for(int i = 0; i < num_blocks; i++) {
        printf("| B%d  | %-10s | %-29s ", blocks[i].id, blocks[i].label, blocks[i].code);
        if(blocks[i].reachable) {
            printf("| REACHABLE%-15s", " ");
            reachable_count++;
        } else {
            printf("| *** UNREACHABLE ***%-3s", " ");
            unreachable_count++;
        }
        printf("|\n");
    }
    printf("+-----+------------+------------------------------+--------------------+\n");
    printf("\n[ANALYSIS]  Total blocks     : %d\n", num_blocks);
    printf("[ANALYSIS]  Reachable        : %d\n", reachable_count);
    printf("[ANALYSIS]  Unreachable      : %d\n", unreachable_count);
}

void optimize_cfg() {
    printf("\n[OPTIMIZER] Blocks eliminated (unreachable):\n");
    for(int i = 0; i < num_blocks; i++) {
        if(!blocks[i].reachable) {
            printf("  [-] B%d  (%-10s) \"%s\"\n", blocks[i].id, blocks[i].label, blocks[i].code);
        }
    }

    printf("\n[RESULT] Optimized CFG (unreachable blocks removed):\n");
    printf("+-----+-------------------------------------------------------------+\n");
    for(int i = 0; i < num_blocks; i++) {
        if(blocks[i].reachable) {
```

```
            printf("| B%d  | %-59s |\n", blocks[i].id, blocks[i].code);
        }
    }
    printf("+-----+-----------------------------------------------------------+\n");
}

int main() {
    printf("================================================================\n");
    printf("  CS-346 Compiler Construction  |  Lab 10: Code Optimization\n");
    printf("  Task 2: Unreachable Code Detection via Control Flow Analysis\n");
    printf("  Source program: sloppy.c\n");
    printf("================================================================\n");

    // Build CFG from sloppy.c
    add_block(0, "ENTRY", "int token_count=100, lc=tc");
    add_block(1, "CONST_FOLD", "buffer=8*1024, max=50*2 ...");
    add_block(2, "CONST_PROP", "symbols=tc/10, kw=tc/25");
    add_block(3, "CSE", "total_a=(tc+lc)*2 x4");
    add_block(4, "STRENGTH", "mem_a=tc*4, mem_b=tc*8 ..");
    add_block(5, "DEAD_CODE", "dead_var1=9999; dead_var2");
    add_block(6, "ALG_SIMP", "alg_a=tc*1 ... alg_e=tc/1");
    add_block(7, "LOOP_BODY", "for i=0..9: parse_sum+=..");
    add_block(8, "COPY_PROP", "cp_x=tc; cp_y=cp_x; cp_z");
    add_block(9, "BRANCH", "if(flag==1) goto end_proc");
    add_block(10, "UNREACH_1", "printf(NEVER prints!)");
    add_block(11, "UNREACH_2", "symbols*=999; kw+=500");
    add_block(12, "END_PROC", "end_processing: label");
    add_block(13, "OUTPUT", "printf(all results)");
    add_block(14, "EXIT", "return 0");

    // Add successor edges
    add_successor(0, 1);
    add_successor(1, 2);
    add_successor(2, 3);
    add_successor(3, 4);
    add_successor(4, 5);
    add_successor(5, 6);
    add_successor(6, 7);
    add_successor(7, 8);
    add_successor(8, 9);
    add_successor(9, 12);      // Always goes to end_proc (flag==1 constant)
    add_successor(10, 11);
    add_successor(11, 12);
    add_successor(12, 13);
    add_successor(13, 14);

    print_cfg();

```

```c
    // Perform DFS from entry block (B0)
    dfs(0);

    print_reachability();
    optimize_cfg();

    printf("\n[DONE] Control flow analysis complete.\n");
    return 0;
}
```

Listing 2: unreachable_detector.c

4.4 Program Output

```
================================================================
  CS-346 Compiler Construction  |  Lab 10: Code Optimization
  Task 2: Unreachable Code Detection via Control Flow Analysis
  Source program: sloppy.c
================================================================

=== Original Control Flow Graph (CFG) ===
+-----+------------+------------------------------+-----------------

| ID  | Label      | Code                         | Successors
       |
+-----+------------+------------------------------+-----------------

| B0  | ENTRY      | int token_count=100, lc=tc   | B1
       |
| B1  | CONST_FOLD | buffer=8*1024, max=50*2 ...  | B2
       |
| B2  | CONST_PROP | symbols=tc/10, kw=tc/25      | B3
       |
| B3  | CSE        | total_a=(tc+lc)*2 x4         | B4
       |
| B4  | STRENGTH   | mem_a=tc*4, mem_b=tc*8 ..    | B5
       |
| B5  | DEAD_CODE  | dead_var1=9999; dead_var2    | B6
       |
| B6  | ALG_SIMP   | alg_a=tc*1 ... alg_e=tc/1    | B7
       |
| B7  | LOOP_BODY  | for i=0..9: parse_sum+=..    | B8
       |
| B8  | COPY_PROP  | cp_x=tc; cp_y=cp_x; cp_z     | B9
       |
| B9  | BRANCH     | if(flag==1) goto end_proc    | B12
       |
| B10 | UNREACH_1  | printf(NEVER prints!)        | B11
       |
| B11 | UNREACH_2  | symbols*=999; kw+=500        | B12
       |
| B12 | END_PROC   | end_processing: label        | B13
       |
| B13 | OUTPUT     | printf(all results)          | B14
       |
| B14 | EXIT       | return 0                     | EXIT
       |
```

```
+-----+------------+------------------------------+-----------------


=== Reachability Analysis Results ===
+-----+------------+------------------------------+-----------------

| ID  | Label      | Code                         | Status
       |
+-----+------------+------------------------------+-----------------

| B0  | ENTRY      | int token_count=100, lc=tc   | REACHABLE
       |
| B1  | CONST_FOLD | buffer=8*1024, max=50*2 ...  | REACHABLE
       |
| B2  | CONST_PROP | symbols=tc/10, kw=tc/25      | REACHABLE
       |
| B3  | CSE        | total_a=(tc+lc)*2 x4         | REACHABLE
       |
| B4  | STRENGTH   | mem_a=tc*4, mem_b=tc*8 ..    | REACHABLE
       |
| B5  | DEAD_CODE  | dead_var1=9999; dead_var2    | REACHABLE
       |
| B6  | ALG_SIMP   | alg_a=tc*1 ... alg_e=tc/1    | REACHABLE
       |
| B7  | LOOP_BODY  | for i=0..9: parse_sum+=..    | REACHABLE
       |
| B8  | COPY_PROP  | cp_x=tc; cp_y=cp_x; cp_z     | REACHABLE
       |
| B9  | BRANCH     | if(flag==1) goto end_proc    | REACHABLE
       |
| B10 | UNREACH_1  | printf(NEVER prints!)        | *** UNREACHABLE
   *** |
| B11 | UNREACH_2  | symbols*=999; kw+=500        | *** UNREACHABLE
   *** |
| B12 | END_PROC   | end_processing: label        | REACHABLE
       |
| B13 | OUTPUT     | printf(all results)          | REACHABLE
       |
| B14 | EXIT       | return 0                     | REACHABLE
       |
+-----+------------+------------------------------+-----------------


[ANALYSIS]  Total blocks    : 15
[ANALYSIS]  Reachable       : 13
[ANALYSIS]  Unreachable     : 2

[OPTIMIZER] Blocks eliminated (unreachable):
  [-] B10  (UNREACH_1 ) "printf(NEVER prints!)"
  [-] B11  (UNREACH_2 ) "symbols*=999; kw+=500"

[RESULT] Optimized CFG (unreachable blocks removed):
+-----+------------------------------------------------------------+
| B0  | int token_count=100, lc=tc                                 |
| B1  | buffer=8*1024, max=50*2 ...                                |
| B2  | symbols=tc/10, kw=tc/25                                    |
| B3  | total_a=(tc+lc)*2 x4                                       |
| B4  | mem_a=tc*4, mem_b=tc*8 ..                                  |
```

```
| B5  | dead_var1=9999; dead_var2                                    |
| B6  | alg_a=tc*1 ... alg_e=tc/1                                    |
| B7  | for i=0..9: parse_sum+=..                                    |
| B8  | cp_x=tc; cp_y=cp_x; cp_z                                     |
| B9  | if(flag==1) goto end_proc                                    |
| B12 | end_processing: label                                        |
| B13 | printf(all results)                                          |
| B14 | return 0                                                     |
+-----+--------------------------------------------------------------+

[DONE] Control flow analysis complete.
```

Listing 3: Output of unreachable_detector.c

4.5 Analysis of Results

- **15 basic blocks** constructed from `sloppy.c`.
- DFS from B0 visits **13 blocks**; B10 and B11 are **never visited**.
- B10 (`printf("NEVER prints!")`) and B11 (`symbols*=999; keywords+=500`) are identified as **unreachable** and eliminated.
- The **optimized CFG** contains 13 blocks with identical observable behaviour.

5 Compilation Instructions

```
# Task 1 -- Sloppy
gcc sloppy.c    -o sloppy    -Wall && ./sloppy

# Task 1 -- Optimized
gcc optimized.c -o optimized -Wall && ./optimized

# Verify identical output
./sloppy > out1.txt && ./optimized > out2.txt && diff out1.txt out2.
   txt
# (no output = identical)

# Task 2 -- Unreachable code detector
gcc unreachable_detector.c -o detector -Wall && ./detector
```

Listing 4: GCC compilation commands

6 Conclusion

This lab demonstrated eight classical code-optimization techniques applied to a realistic C program:

1. **Constant Folding** pre-evaluates `8*1024`→`8192` at compile time.
2. **Constant Propagation** substitutes the known value 100 in place of `token_count`, enabling further simplifications.
3. **Common Subexpression Elimination** computes `(tc+lc)` once via `sum_tc` and reuses it.

4. **Strength Reduction** replaces expensive multiplications with bit-shifts (`«`).
5. **Dead Code Elimination** removes `dead_var1` and `dead_var2` whose computed values are never observed.
6. **Algebraic Simplification** removes identity operations ($x \times 1$, $x + 0$, $x \times 0$, etc.).
7. **Loop Invariant Code Motion** hoists `max_tokens * 2` out of the loop, reducing its execution from 10× to 1×.
8. **Copy Propagation** eliminates intermediate copies `cp_x` and `cp_y`.

For Task 2, a Control Flow Graph was constructed for `sloppy.c` and a DFS-based reachability analysis correctly identified **two unreachable basic blocks** (B10, B11) – the code located after the always-true `if(flag==1)` branch. These blocks were automatically eliminated, producing a cleaner 13-block CFG with identical observable behaviour.

All optimized programs produced **output identical** to the original sloppy program, confirming that the transformations preserve program semantics.

References

[1] Wikipedia. *Optimizing compiler*. https://en.wikipedia.org/wiki/Optimizing_compiler

[2] Wikipedia. *Control-flow graph*. https://en.wikipedia.org/wiki/Control_flow

[3] A.V. Aho, M.S. Lam, R. Sethi, J.D. Ullman. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley, 2006.